Mountain Quest 2D

Geometria procedurale, movimento e collisioni

Esercitazione di Computer Graphics – Durata: 3 ore

C++ • OpenGL • GLFW • GLM

Obiettivo dell'esercitazione

L'obiettivo e' completare lo sviluppo di **Mountain Quest 2D**, un semplice videogioco bidimensionale realizzato in C++ e OpenGL. Un personaggio controllato dall'utente si muove in un paesaggio montano, salta sulle piattaforme e raccoglie cristalli.

L'esercitazione applica le conoscenze gia' acquisite su geometrie, trasformazioni e rendering OpenGL, concentrandosi su tre aspetti:

1. **Geometria procedurale**: generare le montagne con funzioni sinusoidali e la primitiva `GL_TRIANGLE_STRIP`.
2. **Simulazione del movimento**: implementare spostamento orizzontale, salto e gravita'.
3. **Collisioni e interazioni**: rilevare le sovrapposizioni con AABB e gestire le risposte agli urti.

Materiale fornito

Il progetto C++ comprende le classi gia' studiate per geometrie, istanze, matrici di trasformazione, shader e rendering. Alcune funzioni di `Landscape.cpp` e `Game.cpp` contengono i commenti `TODO 1`, `TODO 2`, `TODO 3`.

Prima di iniziare: aprire il progetto, compilarlo ed eseguirlo. Verificare che la finestra OpenGL si apra correttamente. Le funzioni ancora incomplete saranno sviluppate durante l'esercitazione.

TODO 1 — Generazione procedurale delle montagne

File: `Landscape.cpp` **Metodo:** `Landscape::addMountain()`

Ogni montagna e' descritta in coordinate locali da un profilo sinusoidale:

$$\boxed{y(x) = h + A\sin(\omega x + \varphi)}$$

Qui h e' l'altezza media, A l'ampiezza, ω la frequenza angolare e φ la fase. Il profilo e' campionato in $N+1$ punti:

$$x_i = \frac{iL}{N}, \qquad i = 0, \ldots, N,$$

dove L e' la larghezza della montagna. A ogni ascissa corrispondono due vertici:

$$B_i = (x_i, 0), \qquad T_i = (x_i, y(x_i)).$$

La sequenza $B_0, T_0, B_1, T_1, \ldots, B_N, T_N$ definisce la superficie mediante `GL_TRIANGLE_STRIP`.

Attivita' richiesta

Individuare `TODO 1` in `addMountain()` e sostituire il calcolo provvisorio dell'altezza con il campionamento sinusoidale. Compilare ed eseguire il progetto.

Risultato atteso: tre montagne con profili ondulati differenti; gli alberi devono continuare a poggiare sulla collina in primo piano.

TODO 2 — Movimento orizzontale e salto

File: `Game.cpp` **Metodo:** `Game::updatePlayer()`

I tasti `A` e `D` determinano la velocita' orizzontale:

$$\boxed{v_x = (\text{float}(\texttt{right}) - \text{float}(\texttt{left}))V}, \qquad V = 43.$$

Le variabili `left` e `right` sono booleane: `true` viene convertito in 1 e `false` in 0. La posizione orizzontale viene aggiornata a ogni frame:

$$\boxed{x_{n+1} = x_n + v_x \Delta t}.$$

Una nuova pressione di `SPACE`, se il personaggio e' a terra, avvia il salto assegnando la velocita' verticale iniziale $v_y = 75$. Durante il volo agisce la gravita' $g = -135$. Il codice usa l'integrazione di Eulero semi-implicito:

$$\boxed{\begin{aligned} v_y^{n+1} &= v_y^n + g\Delta t, \\ y_{n+1} &= y_n + v_y^{n+1}\Delta t. \end{aligned}}$$

Δt e' il tempo trascorso tra due aggiornamenti consecutivi, in secondi.

Attivita' richiesta

Completare `TODO 2` per:

1. assegnare v_x in funzione dei tasti `A` e `D`;
2. aggiornare la posizione orizzontale usando `dt`;
3. avviare il salto soltanto con una nuova pressione di `SPACE` e con `onGround=true`;
4. aggiornare la velocita' verticale applicando la gravita';
5. aggiornare la quota del personaggio.

Risultato atteso: movimento verso destra e sinistra, salto verticale, salto direzionale e possibilita' di cambiare direzione orizzontale in volo.

TODO 3 — Collision detection e collision response

File: `Game.cpp` **Funzioni:** `intersects()` e `Game::updatePlayer()`

Le collisioni sono approssimate con *Axis-Aligned Bounding Boxes* (AABB), rettangoli allineati agli assi del mondo. Ogni AABB e' definita da centro e semidimensioni:

```
struct AABB {
    glm::vec2 center;
    glm::vec2 halfSize;
};
```

Due AABB A e B si sovrappongono se sono vere *entrambe* le condizioni:

$$\boxed{\begin{aligned} |c_{Ax} - c_{Bx}| &< h_{Ax} + h_{Bx}, \\ |c_{Ay} - c_{By}| &< h_{Ay} + h_{By}. \end{aligned}}$$

Le grandezze sono espresse in coordinate del mondo. Il test geometrico rileva la sovrapposizione (*collision detection*); la correzione di posizione e velocita' costituisce la *collision response*.

Attivita' richiesta

Completare `TODO 3` per:

1. implementare `intersects()`, che restituisce `true` quando le due AABB si sovrappongono;
2. correggere la posizione del personaggio quando urta lateralmente una piattaforma;
3. distinguere atterraggio sulla sommità e urto dal basso;
4. aggiornare la velocita' verticale e `onGround` in seguito all'atterraggio.

Lo stesso test AABB viene utilizzato per riconoscere il contatto con i cristalli: in questo caso non si respinge il personaggio, ma si registra la raccolta e si aggiorna il punteggio.
Risultato atteso: il personaggio non attraversa le piattaforme, puo' atterrare sulla loro sommita', saltare nuovamente e raccogliere i cristalli.

Verifica finale

Al termine, compilare ed eseguire il progetto. Verificare che:

1. le montagne abbiano profili sinusoidali;
2. il personaggio risponda ai tasti `A`, `D` e `SPACE`;
3. il salto risenta della gravita' e sia controllabile orizzontalmente;
4. le piattaforme impediscano l'attraversamento;
5. sia possibile atterrare e saltare nuovamente;
6. il contatto con un cristallo ne attivi la raccolta.